

InterviewOS

AI-Powered Voice Interview Platform

[EchoSphere Agora Conversational AI Hackathon](#)

Two AI interviewers. Live evidence tracking. Real-time voice.
A complete PM mock interview system built on Agora Conversational AI v2.

Built by Kifayat Khan

What It Does

InterviewOS runs a full-length PM interview with two AI interviewers who dynamically decide what to ask based on a live evidence map that tracks claims, evidence, gaps, vague statements, and contradictions across 6 competency areas.

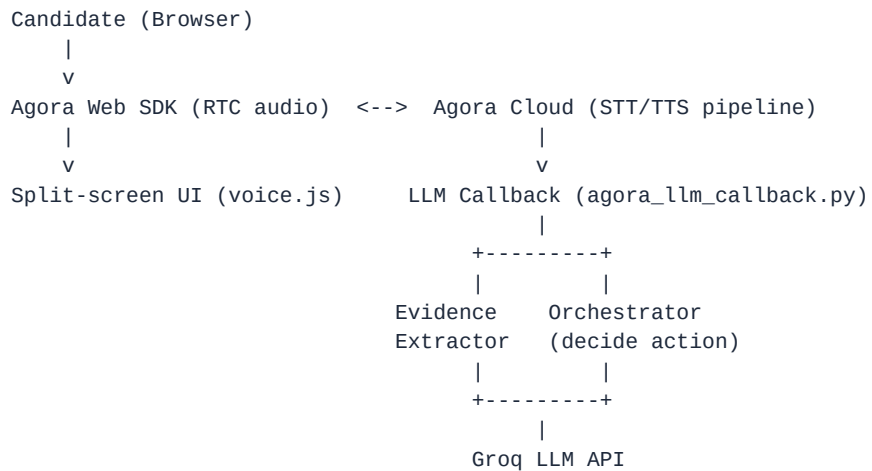
- **Real-Time Voice Interviews** — Agora Conversational AI v2 with Deepgram STT + MiniMax TTS
- **Two Distinct Personas** — Maya (warm Senior PM) and Raj (blunt Hiring Manager) with unique voices
- **Live Evidence Map** — Claims, evidence, gaps, vague statements, contradictions tracked per turn
- **9-Action Orchestrator** — follow_up, probe, challenge, advance, respond, begin_scenario, hand_off, cameo, wrap_up
- **Think-Time Support** — Candidates can request time to think with a visible countdown timer
- **Competency Feedback Report** — Scores, strengths, areas for improvement, hire recommendation

Tech Stack & Tools

Layer	Technology
Backend	Python FastAPI
LLM	Groq API (openai/gpt-oss-20b)
Voice / RTC	Agora Conversational AI v2
STT	Deepgram Nova 3 (managed by Agora)
TTS	MiniMax Speech 2.8 Turbo (managed by Agora)
Frontend	Vanilla HTML/CSS/JS + Agora Web SDK 4.x
Tunnel	ngrok (reserved domain for Agora callbacks)

Agora Conversational AI v2 is the voice middleware at the center of the stack — it manages the full STT/TTS pipeline in the cloud, posting candidate utterances to our custom LLM callback endpoint.

System Architecture



Data Flow

1. Candidate speaks into their microphone via the browser
2. Agora Web SDK streams raw audio to Agora Cloud over RTC
3. Agora Cloud runs Deepgram Nova 3 STT to transcribe the audio
4. Agora POSTs the transcript to our custom LLM callback endpoint
5. Evidence Extractor + Orchestrator run in parallel (`asyncio.gather`)
6. The orchestrator decides the next action from 9 options
7. The response streams back via SSE to Agora Cloud
8. Agora Cloud runs MiniMax TTS and streams audio back to the candidate

Two AI Personas

Role	Senior PM Interviewer	Role	VP/Hiring Manager
Style	Warm, patient, probing	Style	Blunt, skeptical, direct
Voice	English_captivating_female1 (MiniMax)	Voice	English_confident_male1 (MiniMax)
Focus Areas	Product Sense, Customer Understanding, Metrics & Analytics, Prioritization	Focus Areas	Execution & Delivery, Leadership, Communication
Goes First	Yes — opens the interview, builds rapport	Takes Over	After Maya's section via hand_off

Cameo System

Either interviewer can jump in with a quick question during the other's section. The orchestrator triggers a **cameo** action, which swaps the Agora agent to the other persona's voice for one exchange, then swaps back automatically.

Orchestration Engine

Two parallel AI subsystems run on every candidate utterance via `asyncio.gather()`:

Evidence Extractor

Analyzes each candidate response to update the live evidence map with:

- New claims made
- Evidence supporting existing claims
- Gaps in reasoning
- Vague or unsubstantiated statements
- Contradictions

Orchestrator (Action Decider)

Reviews the evidence map and conversation history to pick one of 9 actions:

Action	Description
follow_up	Dig deeper into the current topic
probe	Ask for specific evidence or examples
challenge	Push back on weak claims
advance	Move to a new competency area
respond	Acknowledge and continue
begin_scenario	Start a new scenario question
hand_off	Transfer to the other interviewer (Agora agent swap)
cameo	Quick interjection by the other persona (Agora agent swap + auto-return)
wrap_up	End the interview and generate feedback

The last 3 actions (**hand_off**, **cameo**, **wrap_up**) trigger Agora agent lifecycle operations — stopping the current agent and starting a new one with a different voice and system prompt.

Agora Integration Deep Dive

Agora Conversational AI v2 is not just a voice transport — it's the real-time middleware that makes the entire interview possible.

Custom LLM Callback

Agora POSTs to `/agora/llm/{session_id}` on every candidate utterance. The server responds with an SSE stream of the AI interviewer's reply. This is where deduplication, message merging (2s window), and think-time detection happen.

Agent Lifecycle Management

Server creates, swaps, and stops Agora agents via REST API (`/v2/join`, `/leave`). Each agent carries its own STT/TTS/LLM config. Persona swaps involve: stop current agent → 4s delay → start new agent with different voice + system prompt.

RTC Audio Transport

Agora Web SDK 4.x handles the browser-side audio. Volume-indicator polling with debounce drives the speaking/listening/thinking UI states. Echo suppression uses agent-first priority.

STT + TTS Pipeline

Deepgram Nova 3 for speech-to-text and MiniMax Speech 2.8 Turbo for text-to-speech, both managed entirely by Agora Cloud. The server never touches raw audio.

Voice Interview Lifecycle

Step 1: Session Created

User enters name/email, optionally uploads resume. Server creates interview session with competency map.

Step 2: Voice Started

Browser joins Agora RTC channel. Server calls Agora REST API to start Maya agent with her voice + system prompt.

Step 3: Maya Interviews

Maya covers Product Sense, Customer Understanding, Metrics, Prioritization. Evidence map builds with each turn.

Step 4: Orchestrator Decides

On each turn: Evidence Extractor + Orchestrator run in parallel. Orchestrator picks from 9 actions.

Step 5: Hand-off to Raj

Orchestrator triggers hand_off. Server stops Maya agent, waits 4s, starts Raj agent with his voice + prompt.

Step 6: Raj Interviews

Raj covers Execution, Leadership, Communication. Cameos can occur in either direction.

Step 7: Wrap-up

Orchestrator triggers wrap_up when competency areas are sufficiently covered.

Step 8: Feedback Report

Server generates a competency-based feedback report with scores, strengths, improvements, and hire recommendation.

Project Structure

File	Purpose
main.py	Core interview logic — sessions, prompts, evidence extraction, orchestration, feedback
agora_agent.py	Agora agent lifecycle — start/stop/swap agents via REST API
agora_config.py	Voice session state + Agora config constants + RTC token generation
agora_llm_callback.py	Custom LLM callback — dedup, message merging, think-time, SSE pipeline
agora_routes.py	Voice HTTP endpoints — start/stop/status/ready
static/index.html	Split-screen meeting UI — lobby, participants, transcript
static/voice.js	Agora RTC client, volume indicator, think timer, mute toggle
requirements.txt	Python dependencies